

JEON MINGYU · BACKEND ENGINEER

Redis에서 선점하고 MySQL에서 발급을 확정한 백엔드 개발 경험

CLUTCH에서는 선착순 쿠폰의 재고·발급·복구를 맡았고, Daenggo에서는 게시글·댓글 API와 팀 실행 환경을 만들었습니다.

Java 21

Spring Boot

MySQL

Redis Lua

Kafka

Docker Compose

NAME

전민규

ROLE

Backend Engineer

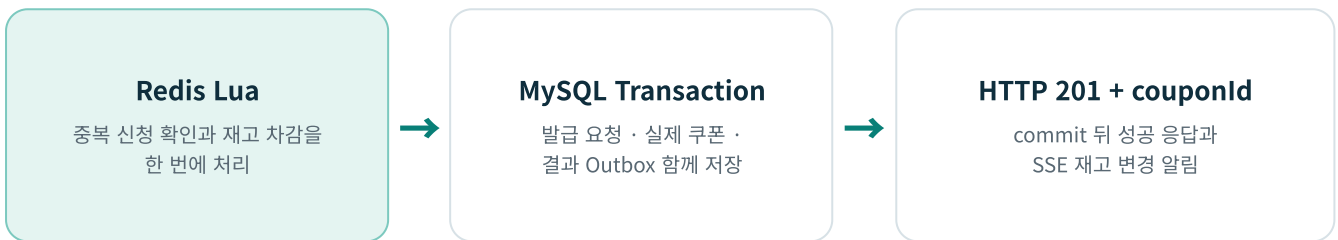
GITHUB

github.com/minggure

FEATURED PROJECT · CLUTCH-BE

Redis가 재고를 잡고, MySQL이 쿠폰 발급을 확정합니다

Redis 재고만 줄어든 뒤 MySQL 저장이 실패하면 성공 응답과 쿠폰함의 상태가 달라집니다. Redis는 발급 자리를 선점하고, **MySQL에 `user_coupon`이 생성돼야** 발급 성공으로 처리했습니다.



1 초과·중복 발급 차단

Redis Lua가 재고와 회차별 당첨자 기록을 함께 바꿔 경쟁 조건을 막습니다.

2 저장 실패 보상

MySQL 저장에 실패하면 Redis 재고와 당첨 기록을 되돌려 다시 신청할 수 있게 합니다.

3 Kafka를 후속 경로로 제한

이미 만든 쿠폰은 Kafka 장애와 관계없이 유지하고, 전달하지 못한 결과는 Outbox에서 다시 발행합니다.

담당 범위

- 쿠폰 이벤트·회차·발급 요청 API
- Redis 재고 초기화와 Lua 동시성 제어
- 실제 쿠폰 생성과 rollback 보상
- Kafka 계약과 Outbox 재시도
- 잔여 재고 API와 SSE 스트림
- Redis 장애 복구와 관리자 API
- DB 잠금 병목 분석과 Redis 선평단
- 대량 데이터 정합성 검사

HTTP 201을 반환하기 전에 실제 쿠폰 ID가 만들어졌는지 확인합니다.

프로젝트 · github.com/seok-cess/Clutch-BE

근거 · PR #24 Redis 동시성 제어 · PR #47 실제 쿠폰 생성 기준 동기 발급

DESIGN FOR FAILURE

Redis 상태가 불확실하면 발급부터 멈춥니다

정확한 재고를 모른 채 MySQL로 우회하면 Redis와 MySQL이 각각 재고를 판단합니다. 연결 장애나 필수 키 누락을 감지하면 HTTP 503으로 요청을 끝내고, MySQL의 실제 쿠폰으로 Redis를 다시 만듭니다.

**UNAVAILABLE · 새 발급 차단**

연결 장애나 필수 키 누락을 감지하면 새 요청을 HTTP 503으로 종료합니다.

**RECOVERING · MySQL 검사**

실제 쿠폰 수와 당첨 사용자 집합을 읽어 성공 결과가 맞는지 확인합니다.

**Redis 데이터 재구축**

재고, 당첨자, 회차 컨텍스트를 Lua로 함께 교체합니다.

**READY · 값이 맞을 때만 재개**

재구축 결과가 기대값과 같을 때만 발급을 다시 엽니다.

재고 100장 장애 실험

20장 발급 → Redis 중단 → 장애 중 503 → 복구 → 남은 80장 발급 → 101번째 요청은 정상 품절

최종 결과**실제 쿠폰 100장**

초과 발급 0건 · 중복 발급 0건

모든 성공 요청이 같은 행에서 기다렸습니다

19/20

Hikari active

178

커넥션 최대 대기

2,914

행 잠금 횟수 / 분

1,115s

잠금 대기 시간 / 분

BEFORE · 요청마다 집계

- 실제 쿠폰 저장
- 동일 `success_count` 행 갱신
- X-lock 앞에서 직렬 대기
- Hikari와 Tomcat까지 적체

**AFTER · 발급과 집계 분리**

- 발급 transaction은 실제 쿠폰만 저장
- 5초마다 한 번 GROUP BY
- 전용 인덱스로 집계 범위 축소
- MySQL named lock으로 중복 실행 차단

DB에 들어오기 전에 끝낸 요청

중복 신청과 품절 요청은 Redis에서 판정해 MySQL에 접근하지 않습니다. 발급 transaction에는 사용자 쿠폰 저장만 남기고, 조회용 성공 수량은 최대 5초의 차이를 허용했습니다.

MEASURE THE SERVER AND THE GENERATOR

20,000건을 모두 애플리케이션 응답으로 끝냈습니다

재고 10,000장에 고유 사용자 20,000명이 60초 동안 증가하는 Ramp 테스트입니다. 요청 20,000건을 같은 순간에 보낸 Burst 결과로 표현하지 않았습니다.

10,000

발급 성공

10,000

정상 품질

0

전송·예상 밖 실패

951ms

Claim p95

Docker k6

- dial: i/o timeout 6,216건
- DB connection pool을 늘려도 동일
- Docker Desktop · WSL NAT 경유
- Claim p95 1.84초



Windows 네이티브 k6

- 서버와 재고·사용자 조건 유지
- 부하 발생기의 NAT 경로 제거
- 전송 실패 0건
- Claim p95 951.19ms

351만 요청 이력을 수정 없이 검사했습니다

1,000,003

사용자

3,517,209

쿠폰 발급 요청

797,209

실제 발급 쿠폰

READ ONLY transaction과 REPEATABLE READ 일관 스냅샷을 사용했습니다. 개인정보는 출력하지 않고 건수·위반 수·fingerprint만 기록했으며, 고아 참조·쿠폰 누락·중복·수량 초과 등 핵심 위반은 0건이었습니다.

관리자 통계 조회

최근 7일 원본 634,917건을 요청마다 GROUP BY하던 구조를 일별 통계 7행 조회로 바꿨습니다.

쿼리 구간: 447ms → 0.0296ms

측정 범위를 함께 기록

0.0296ms는 같은 로컬 MySQL에서 측정한 DB 쿼리 값입니다. 원격 API 전체 응답 속도로 확대하지 않았습니다.

전체 clean build: 558개 테스트 통과

근거 · 2026-08-30 쿠폰 20,000 VU 결과 · 2026-08-31 관리자 일별 통계 검증

기여 PR · #81 Redis 선판단 · #94 관리자 통계 · #111 일별 통계 조회 최적화

SECOND PROJECT · DAENGGO-BE

게시글·댓글 기능을 실제 인증 흐름에 연결했습니다

Daenggo는 반려견 동반 장소 검색, 산책 기록, 커뮤니티를 제공하는 팀 프로젝트입니다. 커뮤니티 백엔드와 팀의 로컬 실행 환경을 맡았습니다.

게시글과 댓글 API

게시글·댓글의 등록, 조회, 수정, 삭제 흐름과 DTO·Entity·Repository 구조를 구현했습니다.

이미지와 조회수

게시글 이미지 등록·편집을 연결하고 조회 요청에서 조회수를 갱신했습니다.

작성자 권한 검증

로그인 사용자와 작성자를 비교해 작성자만 게시글과 댓글을 수정·삭제하도록 막았습니다.

실행 환경 통합

Docker Compose와 Nginx로 프론트엔드, 백엔드, MySQL의 실행 절차를 맞췄습니다.

수정·삭제 요청 흐름



통합 중 고친 문제

임시 사용자 로직이 남아 실제 JWT 인증 사용자와 커뮤니티 작성자가 어긋났습니다.

JWT 오류와 임시 사용자 코드를 정리해 인증 결과를 게시글·댓글 권한 검증에 사용했습니다.

기여 근거

- PR #8 게시물·댓글 구조
- PR #18 이미지·조회수
- PR #23 댓글 기능
- PR #33 JWT 오류 수정

Technology

Java 21

Spring Boot

Spring Security

Spring Data JPA

MySQL

Docker Compose

Nginx

CLUTCH에서는 동시성과 복구를, Daenggo에서는 REST API와 팀 통합을 경험했습니다.